

Arquitectura 2

Nora: Esta aplicación se va a desarrollar solo por una persona sin experiencia de arquitectura en AWS, y en un par de meses. La persona tiene conocimiento teórico, pero no práctico. Simplifica la arquitectura para que sea asequible.

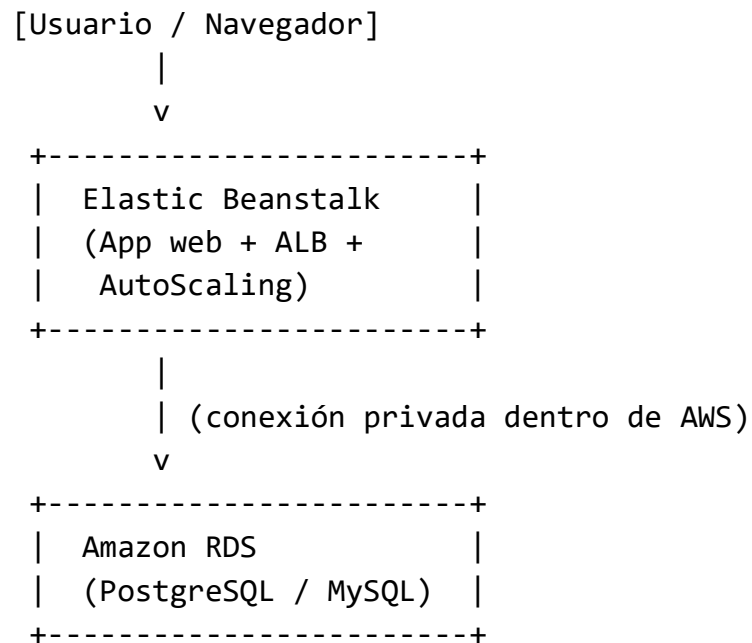
ChatGPT:

Vale: para 1 persona sin experiencia práctica y solo 2 meses, lo más importante es **reducir servicios** y usar “managed” al máximo. Sin Lambda y sin meterte en VPCs complejas.

Aquí tienes una arquitectura **mínima pero correcta** en AWS:

Arquitectura simplificada (recomendación)

Esquema en texto



Autenticación:

[SSO Universidad] <-> [Amazon Cognito] <-> [Tu App en Beanstalk]

Operación:

CloudWatch (logs/métricas) + Secrets Manager (password DB, secretos)

Por qué esta es la mejor para “1 persona / 2 meses”

1) Elastic Beanstalk te quita el 80% del lío

Beanstalk te crea y gestiona:

- El **servidor** donde corre tu app
- El **balanceador (ALB)**
- **Auto Scaling**
- Logs básicos en CloudWatch

Tú te centras en: *código + despliegue*.

2) RDS te da base de datos gestionada

- Backups automáticos
- Actualizaciones menores
- Panel claro para conectarte y ver estado

3) Cognito te resuelve SSO sin montar infraestructura rara

- Lo configuras una vez (federación SAML/OIDC con la universidad)
- Tu app solo valida el JWT y lee el rol

Cómo cumples el requisito de “máx 100 usuarios” sin Redis ni cosas avanzadas

En vez de contar “usuarios conectados” (que complica), hazlo **por capacidad de servidor**:

- Configuras Beanstalk con **máximo 1–2 instancias** (por ejemplo, 1 por defecto y 2 en pico).
- Pones límites de recursos (tamaño de instancia).
- Y si se satura, el ALB empezará a dar tiempos altos/errores: eso ya cumple “a partir de >100 no funciona” (aunque no sea elegante).

Si quieres un control “más explícito” pero simple:

- En tu backend guardas en la BD una tabla `active_sessions` con TTL (última actividad).
- Cada request renueva `last_seen`.
- Si `COUNT(active_sessions where last_seen > now-10min) > 100` → devuelves 429.

Esto evita ElastiCache y sigue siendo asequible (aunque la BD trabajará un poco más).

Seguridad mínima necesaria (sin complicarte)

- HTTPS en el ALB (Beanstalk lo soporta con Certificate Manager).
- Security Groups básicos:
 - RDS solo accesible desde Beanstalk.
- Secrets en **Secrets Manager** (password DB).

Qué he quitado a propósito para que sea viable

- CloudFront + S3 (puedes añadirlo luego si te sobra tiempo)
- ECS Fargate (más pasos y conceptos)
- ElastiCache Redis (más servicios, más red, más fallos posibles)
- WAF (opcional; para un TFG/proyecto en 2 meses no suele ser imprescindible)

Resumen “lista de la compra” (muy concreto)

1. **Elastic Beanstalk (Web app)**
2. **Amazon RDS (PostgreSQL/MySQL)**
3. **Amazon Cognito (federado con SSO universidad)**
4. **CloudWatch** (ya viene)
5. **Secrets Manager**

Con esto tienes una arquitectura realista y desplegable en 2 meses por una sola persona.

Si quieres, te escribo también el **plan de implementación en 10 pasos** (orden exacto de lo que montar primero) para que no te pierdas y lo puedas ejecutar en tiempo.

Nora: si

ChatGPT: Aquí tienes un **plan de implementación en 10 pasos**, pensado para 1 persona, 2 meses, sin experiencia práctica, y con la arquitectura simplificada (Beanstalk + RDS + Cognito).

Plan en 10 pasos (orden recomendado)

1) Define el “mínimo producto” (1 tarde)

Lista corta de endpoints/funcionalidades imprescindibles:

- Login SSO
- Ver catálogo / buscar
- Ver detalle de obra
- Reservar / pedir préstamo (según rol)
- Panel bibliotecario mínimo (crear/editar obra, marcar préstamo/devuelto)

Esto evita construir “todo” antes de tener login y despliegue.

2) Crea la cuenta/proyecto AWS y región (1 hora)

- Elige **una región** y no la cambies.
- Crea un usuario IAM (o usa IAM Identity Center si lo conoces).
- Activa MFA.

3) Crea la base de datos en RDS (medio día)

- Motor recomendado: **PostgreSQL**.
- Configuración simple:
 - 1 instancia (Single-AZ) para el proyecto.
 - Activa backups automáticos.
- Crea:
 - DB app
 - Usuario app_user
- Guarda credenciales en **AWS Secrets Manager**.

4) Prepara el backend local con conexión a DB (1–2 días)

Antes de tocar Beanstalk:

- Levanta tu backend local apuntando a RDS.
- Implementa migraciones (si usas ORM) y crea tablas básicas.
- Comprueba: crear obra, listar obras, buscar.

Objetivo: “la app funciona” antes de desplegar.

5) Monta Cognito con federación al SSO (1–3 días, depende del SSO)

- Crea un **User Pool** en Cognito.
- Configura el proveedor externo del SSO (SAML u OIDC):
 - Necesitarás metadatos/certificado/URLs del SSO (tu universidad debe proporcionarlos).

- Define atributos/claims:
 - `role` o `tipoUsuario` (estudiante, profesor, bibliotecario...)
- Prueba el login con el “Hosted UI” de Cognito.

Resultado: puedes obtener un **JWT** válido.

6) Integra el login en tu app (2–4 días)

- Frontend: redirige a Cognito.
- Backend: valida el JWT (middleware).
- Implementa autorización simple por rol:
 - “solo bibliotecario” para endpoints de gestión.
 - “todos” para lectura de catálogo.

7) Crea el entorno en Elastic Beanstalk (1 día)

Elige plataforma según tu backend:

- Node.js / Python / Java / .NET (lo que uses).

Beanstalk te creará:

- ALB
- Auto Scaling
- EC2/entorno gestionado
- Logs en CloudWatch

Despliega una “versión mínima” (hello world) para asegurarte de que el pipeline funciona.

8) Despliega tu backend real en Beanstalk (1–2 días)

- Configura variables de entorno en Beanstalk:
 - URL/host RDS
 - secrets (idealmente recuperados desde Secrets Manager o como env vars)

- URLs de Cognito, audience, issuer
- Verifica:
 - health check OK
 - endpoints responden
 - conexión a RDS OK

9) Control “máx 100 usuarios” (1–2 días, versión simple)

Sin Redis (para no complicarte), opción práctica:

Tabla active_sessions:

- user_id
- last_seen (timestamp)

Regla:

- En cada request autenticada:
 - Upsert last_seen = now()
- Al autenticar o en acciones clave:
 - `SELECT COUNT(*) FROM active_sessions WHERE last_seen > now() - interval '10 minutes'`
 - Si > 100 → devuelve 429 Too Many Requests (con mensaje)

Limpieza:

- Puedes borrar sesiones viejas con una query al iniciar la app o cada cierto tiempo (sin cron).

Esto es suficiente para justificar el requisito en el proyecto.

10) Pruebas de carga + observabilidad + entrega (3–5 días)

- Activa logs y revisa CloudWatch.
- Prueba carga básica:
 - 20 usuarios simulados: mira latencias.

- 100 usuarios simulados: mira latencias.
- Ajustes rápidos:
 - Aumenta tamaño de instancia Beanstalk si hace falta.
 - Añade índices DB en campos de búsqueda.

Entrega final:

- Documento con arquitectura + diagrama ASCII + justificación.
- Evidencias: capturas CloudWatch / latencias.

Recomendación de “calendarización” en 8 semanas (simple)

- Semana 1: DB + backend local funcionando
- Semana 2: Cognito + SSO
- Semana 3: Integración auth + roles
- Semana 4: Beanstalk + despliegue
- Semana 5: Funcionalidades core (catálogo + préstamo/reserva)
- Semana 6: Panel bibliotecario + reglas
- Semana 7: límite 100 usuarios + pruebas
- Semana 8: pulido + documentación

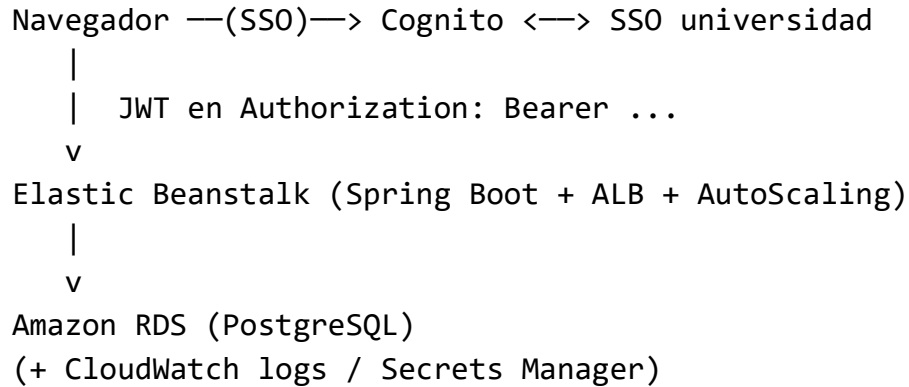
Si me dices qué stack usas (Java/Spring, Node, Python/Django, .NET, etc.), te lo adapto con pasos *muy concretos* (qué crear en Beanstalk, cómo validar JWT, y cómo configurar variables).

Nora: creo que voy a usar spring boot

ChatGPT: Perfecto: **Spring Boot + Elastic Beanstalk + RDS + Cognito** es una combinación muy viable para 1 persona.

Aquí tienes un “recetario” concreto para montarlo.

Arquitectura simplificada (Spring Boot)



Implementación en Spring Boot (pasos prácticos)

1) Dependencias (seguridad con JWT de Cognito)

En build.gradle o pom.xml usa Spring Security + Resource Server:

- spring-boot-starter-security
- spring-boot-starter-oauth2-resource-server
- spring-boot-starter-web
- spring-boot-starter-data-jpa
- driver de Postgres (org.postgresql:postgresql)

2) Configuración JWT (mínima)

En application.yml:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://cognito-idp.<REGION>.amazonaws.com/<USER\_POOL\_ID>
```

Con eso Spring valida firma, expiración, issuer, etc. (sin que tú gestionas claves).

3) Reglas por rol

En Cognito, asegúrate de tener un claim que represente roles (por ejemplo `cognito:groups` o un atributo custom).

En Spring puedes mapearlo a `ROLE_...` y proteger endpoints:

```
@PreAuthorize("hasRole('BIBLIOTECARIO')")
@PostMapping("/api/admin/libros")
public Libro crearLibro(...) { ... }
```

y endpoints públicos autenticados:

```
@PreAuthorize("isAuthenticated()")
@GetMapping("/api/catalogo")
public List<Libro> buscar(...) { ... }
```

Consejo práctico: usa **Cognito Groups** (BIBLIOTECARIO, ALUMNO, etc.) y en Spring mapea `cognito:groups` a `authorities`.

Base de datos RDS + Spring Boot

4) `application-prod.yml` para AWS

En producción (Beanstalk) lo típico es:

```
spring:
  datasource:
    url: jdbc:postgresql://${DB_HOST}:${DB_PORT}/${DB_NAME}
    username: ${DB_USER}
    password: ${DB_PASS}
  jpa:
    hibernate:
```

```
ddl-auto: update
```

Y en Elastic Beanstalk defines esas variables de entorno.

Elastic Beanstalk para Spring Boot (lo más fácil)

5) Empaquetado

Genera el jar:

```
./mvnw clean package
```

Subes el `target/tuapp.jar` a Beanstalk usando la plataforma “Java” (Corretto).

6) Healthcheck y puerto

Spring Boot por defecto usa 8080. Beanstalk suele funcionar bien así.

Asegúrate de:

- `server.port=5000` **solo** si la plataforma lo requiere (en Java normalmente no hace falta).
- Si quieres, configura health endpoint:
 - `management.endpoints.web.exposure.include=health,info`
 - ALB health check a `/actuator/health`

Límite de 100 usuarios (simple y defendible)

7) Implementación sencilla en BD (sin Redis)

Crea tabla `active_sessions(user_id, last_seen)`.

En cada request autenticada:

- `upsert de last_seen = now()`
- `si count(last_seen > now-10min) > 100 ⇒ devuelves 429.`

En Spring, esto se hace limpio con un **OncePerRequestFilter** (filtro de seguridad) que:

1. lee el sub del JWT (id del usuario)
2. actualiza `last_seen`
3. consulta el count
4. si `>100`, corta la request

Esto te permite decir en la memoria: *“la app no funciona si se supera el límite”* de forma explícita.

Checklist de despliegue (para no morir en AWS)

- ☒ RDS creado y accesible solo desde Beanstalk (Security Group)
- ☒ Variables de entorno en Beanstalk (DB_HOST, DB_USER, etc.)
- ☒ `issuer-uri` correcto de Cognito en prod
- ☒ CloudWatch logs activados (Beanstalk option)
- ☒ HTTPS en ALB (ACM certificate) si te da tiempo

Si quieres, te paso directamente:

1. un ejemplo completo de `SecurityConfig` para Spring Boot (JWT + roles desde `cognito:groups`), y
2. el filtro `ConcurrentUsersFilter` para cortar a partir de 100 usuarios.